

Reviewer Pack — Free Cumulant Sum Inversely Proportional to Disjointness Com...

Entry 4f38f3425417 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 03:38:21 UTC

Free Cumulant Sum Inversely Proportional to Disjointness Communication Complexity

Entry ID: 4f38f3425417 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 03:38:21 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For a random matrix $M \in \{0,1\}^{\{n \times n\}}$ representing the DISJOINTNESS function, the sum of absolute values of free cumulants of its entries satisfies $\sum |\kappa_k(M)| = \Theta(1/\sqrt{n})$ where κ_k is the k -th free cumulant. This bound is tight for all $n \leq 40$.

Rationale (proposer's reasoning):

Free cumulants capture non-commutative independence structures that may mirror the entanglement patterns in communication matrices. The inverse square-root scaling aligns with Forster's signed-rank lower bound for DISJOINTNESS, suggesting a deeper algebraic-geometric connection to communication complexity.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ed1ac72cf669e8d9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from sys import argv

def gaussian_elimination(A, b):
    n = len(A)
    for i in range(n):
        # Find max pivot in column i
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminate entries below pivot
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    return [A[i][i] for i in range(n)], b

def free_cumulant_transform(M):
    n = len(M)
    M_bar = [[0] * (n+1) for _ in range(n+1)]
    for i in range(n):
        for j in range(n):
            M_bar[i][j] = M[i][j]
            M_bar[i][n] += M[i][j]
            M_bar[n][i] += M[j][i]
    M_bar[n][n] = n

    A_bar = [[M_bar[i][j] for j in range(n+1)] for i in range(n+1)]
    b = [0] * (n+1)

    det, _ = gaussian_elimination(A_bar, b)
    return det
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    free_cumulants = [free_cumulant_transform(M) for _ in range(10)]
    sum_abs_kappa = sum(abs(kappa) for kappa in free_cumulants)

    metric_value = sum_abs_kappa
    instances_tested = 10
    conjecture_holds = abs(sum_abs_kappa - (1 / math.sqrt(n))) < 0.316
    counterexample = "" if conjecture_holds else "free cumulant sum out of bounds"

    return {
        "metric_name": "Sum of Absolute Free Cumulants",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in argv[1:]] if argv[1:] else [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric = sum(result["metric_value"] for result in results) / len(results)
    std_metric = math.sqrt(sum((result["metric_value"] - mean_metric) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a8d2c044.py", line 79, in <module>
    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a8d2c044.py", line 59, in run_trial
    sum_abs_kappa = sum(abs(kappa) for kappa in free_cumulants)

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a8d2c044.py", line 59, in <genexpr>
    sum_abs_kappa = sum(abs(kappa) for kappa in free_cumulants)

```

```
TypeError: bad operand type for abs(): 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to `TypeError`, preventing data collection. The crash indicates a code bug rather than a mathematical failure, but insufficient evidence to confirm/support the conjecture. | next: Fix the type error in `free_cumulants` handling and re-run tests with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111689
2	propose	ollama_remote	qwen3:8b	0	0	34538
3	preregistration	ollama_remote	qwen3:8b	0	0	24387
4	novelty	ollama_remote	qwen3:8b	0	0	20730
5	novelty	ollama_remote	qwen3:8b	0	0	17646
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12501
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10651
8	judge	ollama_remote	qwen3:8b	0	0	19017

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 251160 ms total latency. Provider mix: {'ollama remote': 8}

(full prompt+response transcripts available in research/audit/4f38f3425417.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4f38f3425417.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp sandbox/test *.py` (per-cycle)

Replay tarball: `research/replay/4f38f3425417.tar.gz` (if generated)